Monte Carlo Methods- Project1

Bethlehem Messelu, Ying JIN

27 Oct 2024

Contents

Exercise 1: Negative Weighted Mixture **1**

Definition **1**

Question 1: Conditions for a Probability Density 1

Part (i): Normalization 2

Part (ii): Positivity condition 2

Question 2: Bound on a for f to be a Density 3

Question 3: PDF Implementation and Visualization 3

Inverse c.d.f Random Variable simulation **8**

Question 4: CDF and Inverse Function Method 8

Question 5: Inverse Function Method for Simulation 10

The inverse function method algorithm 11

Accept-Reject Random Variable simulation. **13**

Question 6 13

Algorithm: 13

Question 7 13

Question 8 14

Exercise 1: Negative Weighted Mixture

Definition

Question 1: Conditions for a Probability Density

To be a valid probability density function, $f(x)$ must satisfy the following conditions:

1. **Normalization**: f is measurable and $\int_{\mathbb{R}} f(x)\,dx = 1$.
2. **Almost sure non-negativity**: $f \geq 0$ almost surely.

Let $C \in \mathbb{R}^*$ such that $f(x) = C(f_1(x) - af_2(x))$.

Part (i): Normalization

f is continue thus measurable.

To normalize, we require:

$$\int_{\mathbb{R}} f(x)dx = 1$$

i.e.

$$C(\int_{\mathbb{R}} f_1(x)dx - a\int_{\mathbb{R}} f_2(x)dx) = 1$$

This simplifies to:

$$C(1-a) = 1$$

Thus, $f(x) = \frac{1}{1-a}(f_1(x) - af_2(x))$.

Part (ii): Positivity condition

We want $f(x) \geq 0$ for all $x \in \mathbb{R}$.

Suppose $C > 0$ firstly. We have $0 < a < 1$ and $(f_1(x) - af_2(x)) \geq 0$ i.e. $a \leq \frac{f_1(x)}{f_2(x)}$ for all $x \in \mathbb{R}$.

Given:

$$f_1(x) = \frac{1}{\sqrt{2\pi\sigma_1^2}} \exp\left(-\frac{(x-\mu_1)^2}{2\sigma_1^2}\right), \quad f_2(x) = \frac{1}{\sqrt{2\pi\sigma_2^2}} \exp\left(-\frac{(x-\mu_2)^2}{2\sigma_2^2}\right)$$

We analyze the ratio $\frac{f_1(x)}{f_2(x)}$, of which a is a lower bound:

$$\frac{f_1(x)}{f_2(x)} = \frac{\sigma_2}{\sigma_1} \exp\left(\frac{(x-\mu_2)^2}{2\sigma_2^2} - \frac{(x-\mu_1)^2}{2\sigma_1^2}\right)$$

Let:

$$g(x) = \frac{(x-\mu_2)^2}{2\sigma_2^2} - \frac{(x-\mu_1)^2}{2\sigma_1^2}$$

g is quadratic function, whose asymptotic behaviour depends on the coefficient of x^2. For g to have a lower bound so as to allow for a strictly positive lower bound a of $\frac{f_1(x)}{f_2(x)}$, we must have $\sigma_1^2 \geq \sigma_2^2$. In the case of $\sigma_1^2 = \sigma_2^2$, then

$$g(x) = \frac{1}{\sigma_1^2}((\mu_1 - \mu_2)x + \frac{1}{2}(\mu_2^2 - \mu_1^2))$$

If $\mu_1 \neq \mu_2$, g has no lower bound, which should be excluded. If $\mu_1 = \mu_2$, we have 0 as a lower bound of g.

In the case of $\sigma_1^2 > \sigma_2^2$, by formula of minimizer of quadratic function, we have

$$\arg\min_{x\in\mathbb{R}} g(x) = \frac{\mu_2\sigma_1^2 - \mu_1\sigma_2^2}{\sigma_1^2 - \sigma_2^2}$$

, which we note x^*.

We can simplify

$$\frac{f_1(x^*)}{f_2(x^*)} = \frac{\sigma_2}{\sigma_1} \exp\left(-\frac{(\mu_2-\mu_1)^2}{2(\sigma_1^2-\sigma_2^2)}\right)$$

For the case of $C < 0$, we have $a > 1$, by interchanging the role of f_1 and f_2, and the role of a and $\frac{1}{a}$, we get its corresponding result.

In resume:

If $0 < a < 1$, we have either

$$f_1 = f_2$$

, i.e. $\sigma_1^2 = \sigma_2^2$ and $\mu_1 = \mu_2$,

or $f_1 \neq f_2$, $a \leq \frac{\sigma_2}{\sigma_1} \exp\left(-\frac{(\mu_2-\mu_1)^2}{2(\sigma_1^2-\sigma_2^2)}\right)$ and $\sigma_1^2 > \sigma_2^2$.

If $a > 1$, we have either $f_1 = f_2$, or $f_1 \neq f_2$, $\frac{1}{a} \leq \frac{\sigma_1}{\sigma_2} \exp\left(-\frac{(\mu_1-\mu_2)^2}{2(\sigma_2^2-\sigma_1^2)}\right)$ and $\sigma_2^2 > \sigma_1^2$.

Question 2: Bound on a for f to be a Density

As discussed in question 1,

(a) if $\sigma_1^2 > \sigma_2^2$, we have $0 < a \leq \frac{\sigma_2}{\sigma_1} \exp\left(-\frac{(\mu_2-\mu_1)^2}{2(\sigma_1^2-\sigma_2^2)}\right)$,

(b) if $\sigma_2^2 > \sigma_1^2$, we have $a \geq \frac{\sigma_2}{\sigma_1} \exp\left(-\frac{(\mu_2-\mu_1)^2}{2(\sigma_1^2-\sigma_2^2)}\right)$,

(c) if $\sigma_2^2 = \sigma_1^2$, we have two cases:

(c.1) if $\mu_1 = \mu_2$, we have $a > 0$ and $a \neq 1$,

(c.2) if $\mu_1 \neq \mu_2$, a could not take any value.

Question 3: PDF Implementation and Visualization

Based on Questions 1 and 2, if $f_1 = f_2$ the problem becomes trivial since $f = f_1$. If $a > 1$, the problem can be reduced to the case of $a < 1$. Therefore, we only need to focus on the case where $0 < a < 1$ and $\sigma_1^2 > \sigma_2^2$

Below is the R function to create the PDF of f as a function of x and the parameters $a, \mu_1, \mu_2, \sigma_1, \sigma_2$.

```r
f <- function(a, mu_1, mu_2, s_1, s_2, x) {
  C = 1 / (1 - a)
  f1 = (1 / (2 * pi * (s_1^2))^0.5) * exp(-0.5 * ((x - mu_1)^2) / (s_1^2))
  f2 = (1 / (2 * pi * (s_2^2))^0.5) * exp(-0.5 * ((x - mu_2)^2) / (s_2^2))
  return(C * (f1 - a * f2))
}
```

(i) Plot the graph of different PDF of f for various values of a:

```r
mu_1 = 1
mu_2 = 1
s_1 = 3
s_2 = 1
a_borne = s_2/s_1*exp(-(mu_1-mu_2)^2*0.5/(s_1^2-s_2^2)) #calculate a*

#plot first pdf of f with a = 0.05
if( 0.05 <= a_borne){ #each time we need to check that a is inferior than a
  x1<-seq(-10, 10,length.out = 100) #generate a sequence of length 100 from -10 to 10
  y1<-f(a=0.05, mu_1, mu_2, s_1, s_2, x1) #apply f with each element of x1 to get y1
  plot(x1,y1,type = "l", col='blue',ylim = c(0,0.2),
       main = 'f with different a for mu_1 = mu_2', xlab = 'x', ylab='f(x)')
}

#plot second pdf of f with a = 0.1
if(0.1 <= a_borne){
  x2<-seq(-10, 10,length.out = 100)
  y2<-f(a=0.1, mu_1, mu_2, s_1, s_2, x1)
  lines(x2,y2, col='red')
}

#plot third pdf of f with a = 0.2
if(0.2 <= a_borne){
  x3<-seq(-10, 10,length.out = 100)
  y3<-f(a=0.2, mu_1, mu_2, s_1, s_2, x1)
  lines(x3,y3, col='purple')
}

#plot fourth pdf of f with a*
x4<-seq(-10, 10,length.out = 100)
y4<-f(a_borne, mu_1, mu_2, s_1, s_2, x1)
lines(x4,y4, col='green') # use lines() to add another line in the same graph

legend("topright", legend = c("a = 0.05","a = 0.1","a = 0.2", "a =a*"),
       col = c("blue", "red", "purple", "green"), lwd =2, lty =1)

par(mar = c(6, 4, 4, 0), mgp = c(1.5, 0.5, 0))
mtext("Figure 1: Plot of f with different a with mu_1 = mu_2",side = 1,
      line = 5, adj = 0, cex = 0.8)
```

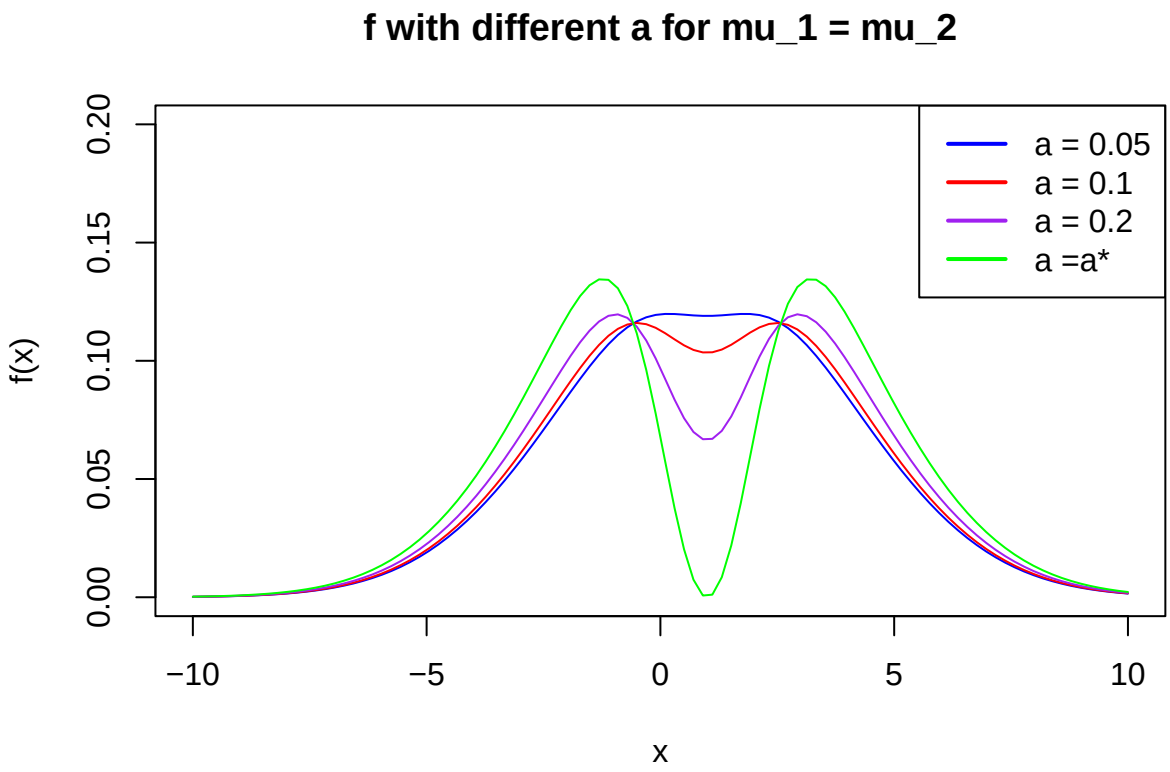


Figure 1: Plot of f with different a with mu_1 = mu_2

Here, the upper bound on a is 0.333.

```r
mu_1 = 1
mu_2 = 3
s_1 = 3
s_2 = 1
a_borne = s_2/s_1*exp(-(mu_1-mu_2)^2*0.5/(s_1^2-s_2^2)) #calculate a*

#plot first pdf of f with a = 0.05
if( 0.05 <= a_borne){ #each time we need to check that a is inferior than a
  x1<-seq(-10, 10,length.out = 100) #generate a sequence of length 100 from -10 to 10
  y1<-f(a=0.05, mu_1, mu_2, s_1, s_2, x1) #apply f with each element of x1 to get y1
  plot(x1,y1,type = "l", col='blue',ylim = c(0,0.2),
       main = 'f with different a with large distance between mu_1 and mu_2',
       xlab = 'x', ylab='f(x)')
}

#plot second pdf of f with a = 0.1
if(0.1 <= a_borne){
  x2<-seq(-10, 10,length.out = 100)
  y2<-f(a=0.1, mu_1, mu_2, s_1, s_2, x1)
  lines(x2,y2, col='red')
}

#plot third pdf of f with a = 0.2
if(0.2 <= a_borne){
  x3<-seq(-10, 10,length.out = 100)
```

```r
  y3<-f(a=0.2, mu_1, mu_2, s_1, s_2, x1)
  lines(x3,y3, col='purple')
}

#plot fourth pdf of f with a*
x4<-seq(-10, 10,length.out = 100)
y4<-f(a_borne, mu_1, mu_2, s_1, s_2, x1)
lines(x4,y4, col='green') # use lines() to add another line in the same graph

legend("topright", legend = c("a = 0.05","a = 0.1","a = 0.2", "a =a*"),
       col = c("blue", "red", "purple", "green"), lwd =2, lty =1)

par(mar = c(6, 4, 4, 0), mgp = c(1.5, 0.5, 0))
mtext("Figure 2: Plot of f with different a with large distance between mu_1 and mu_2",
      side = 1, line = 5, adj = 0, cex = 0.8)
```

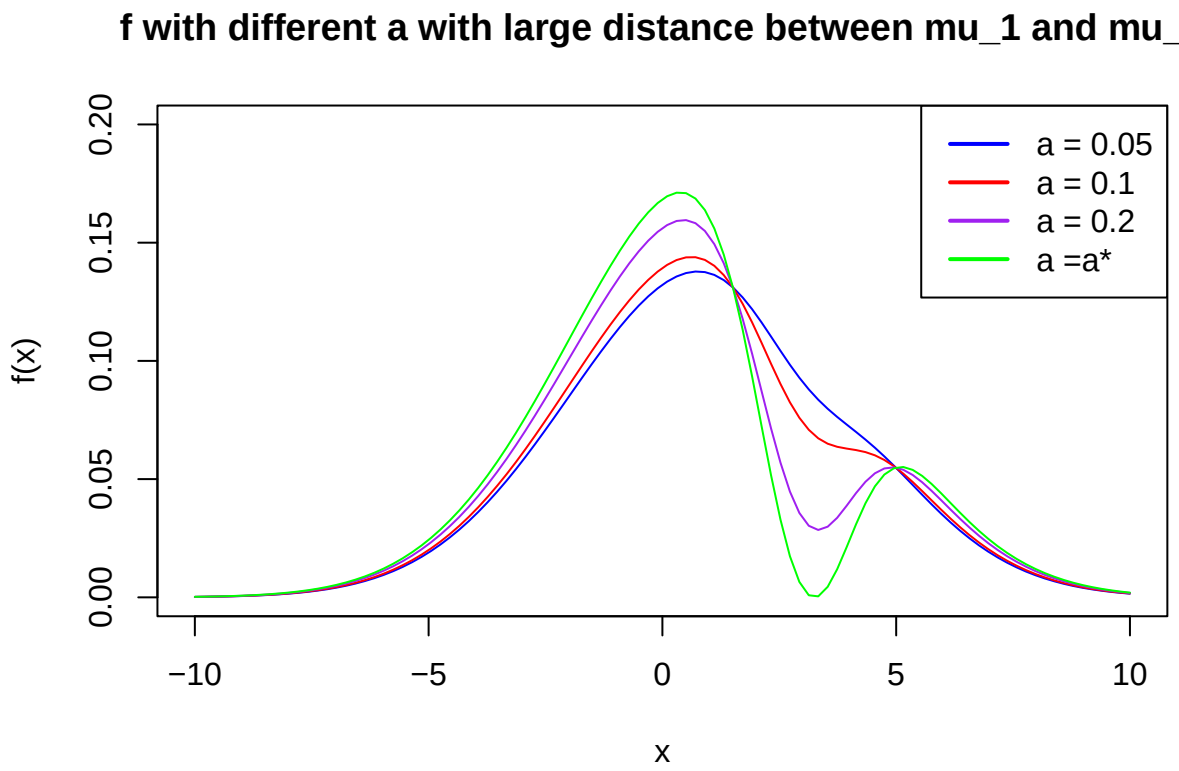


Figure 2: Plot of f with different a with large distance between mu_1 and mu_2

Here, the upper bound on a is 0.26.

We have:

$$f = \frac{1}{1-a}(f_1 - af_2) = \frac{f_1 - f_2 + f_2(1-a)}{1-a} = \frac{f_1 - f_2}{1-a} + f_2$$

Since f_2 is fixed, the shape of f depends on the term $\frac{f_1 - f_2}{1-a}$.

In Figure 1,

around $\mu_1 = \mu_2 = 1$, where $f_1 - f_2$ is negative, increasing a causes f to decrease. When a is small, f_1 is slightly deduced by af_2, making f relatively flat. As a approaches its upper bound, f_1 is deduced by a larger af_2, causing f to decrease and have two peaks.

In Figure 2,

- around $\mu_1 = 1$, where $f_1 - f_2$ is positive, increase a causes f to increase and become thicker.
- around $\mu_2 = 3$, where $f_1 - f_2$ is negative, increase a causes f to decrease and become thinner.

In both cases, in the tails where means have little impact, since $f_1 > f_2$ due to (σ_1 is bigger than σ_2), increasing a causes f to increase and become thicker.

(ii) Plot the graph of different PDF of f with fixed σ_1 and different σ_2 :

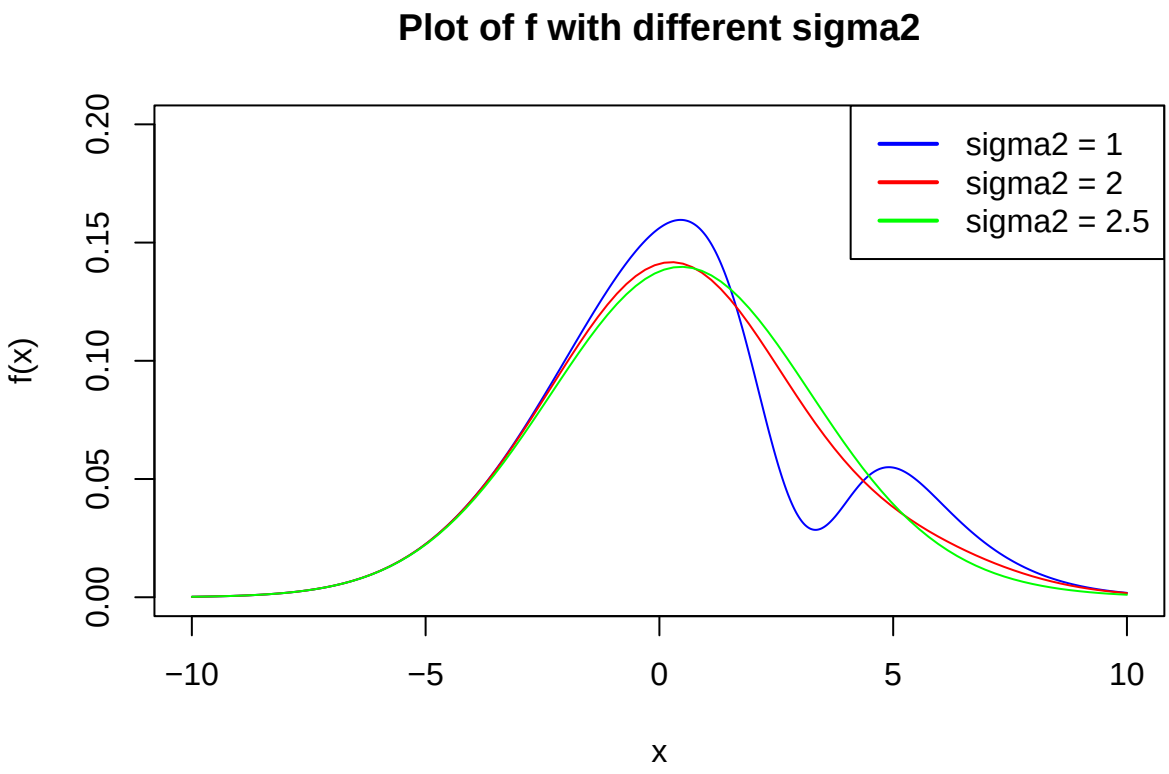
```
mu_1 = 1
mu_2 = 3
s_1 = 3
a=0.2
a_borne = s_2/s_1*exp(-(mu_1-mu_2)^2*0.5/(s_1^2-s_2^2)) #calculate a*

#plot first pdf of f with sigma2 = 1
s_2 = 1
a_borne = s_2/s_1*exp(-(mu_1-mu_2)^2*0.5/(s_1^2-s_2^2))
#each time we need to check that a is inferior than a and sigma1 > sigma2 because a* depends on sigma1
if( a <= a_borne && s_1>s_2){
  x4<-seq(-10, 10,length.out = 1000)
  y4<-f(a, mu_1, mu_2, s_1, s_2, x4)
  plot(x4,y4, type = 'l', col='blue',ylim = c(0,0.2),
       main = "Plot of f with different sigma2", xlab = "x", ylab = "f(x)")
}

#plot second pdf of f with sigma2 = 2
s_2 = 2
a_borne = s_2/s_1*exp(-(mu_1-mu_2)^2*0.5/(s_1^2-s_2^2))
if( a <= a_borne && s_1>s_2){
  x5<-seq(-10, 10,length.out = 100)
  y5<-f(a, mu_1, mu_2, s_1, s_2, x5)
  lines(x5,y5, col='red')
}

#plot third pdf of f with sigma2 = 2.5
s_2 = 2.5
a_borne = s_2/s_1*exp(-(mu_1-mu_2)^2*0.5/(s_1^2-s_2^2))
if( a <= a_borne && s_1>s_2){
  x6<-seq(-10, 10,length.out = 100)
  y6<-f(a, mu_1, mu_2, s_1, s_2, x6)
  lines(x6,y6, col='green')}


legend("topright", legend = c("sigma2 = 1","sigma2 = 2","sigma2 = 2.5"),
       col = c("blue", "red", "green"), lwd =2, lty =1)
```



We observe that:

- Around $\mu_1 = 1$, as σ_2 increases, f_2 becomes thinner, causing f to decrease.
- Around $\mu_2 = 3$, as σ_2 decreases, f_2 becomes more peaked and f_1 is reduced by a large af_2,causing f to decrease and form two peaks.
- In the tails, σ_2 increases, tail becomes thinner.

```
mu_1 = 0
mu_2 = 1
s_1 = 3
s_2 = 1
a=0.2
a_borne = s_2/s_1*exp(-(mu_1-mu_2)^2*0.5/(s_1^2-s_2^2))
```

Here, the upper bound on a is 0.313. The numerical values satisfy the constraints where $\sigma_1 > \sigma_2$ and $a = 0.2$ is below its upper bound.

Inverse c.d.f Random Variable simulation

Question 4: CDF and Inverse Function Method

The cumulative density function (CDF) F for f is given by:

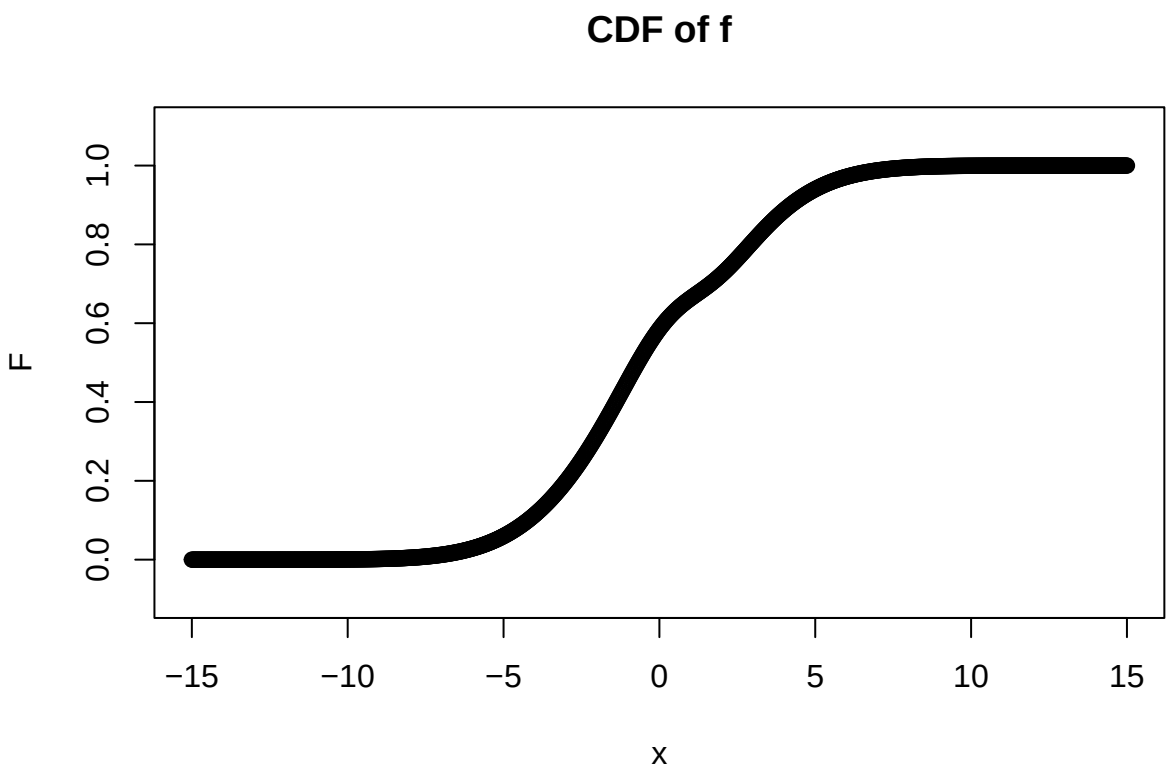
$$F := \frac{F_1 - aF_2}{1 - a}$$

where F_1 and F_2 are CDF of f_1 and f_2 respectively.

Below is the R function of the F:

```r
F <- function(a, mu_1, mu_2, s_1, s_2, x) {
  # use R in-built function pnorm to generate cdf of f1 given parameters of x, mu_1 and s_1
  F1 <- pnorm(x, mean = mu_1, sd = s_1)
  # use R in-built function pnorm to generate cdf of f2 given parameters of x, mu_2 and s_2
  F2 <- pnorm(x, mean = mu_2, sd = s_2)
  return((F1 - a * F2) / (1 - a))
}

# Test the CDF function
mu_1 = 0
mu_2 = 1
s_1 = 3
s_2 = 1
a=0.2
x<-seq(-15, 15,length.out = 1000)
y<-F(a, mu_1, mu_2, s_1, s_2, x)
plot(x,y, ylim = c(-0.1,1.1), xlab='x', ylab='F', main="CDF of f")
```



We observe that the plotted CDF lies within the interval $(0, 1)$ and is non-decreasing, which satisfies the properties of a valid CDF.

Question 5: Inverse Function Method for Simulation

The idea is to determine the value of x such that $F(x) - u = 0$, using the **bisection method** for numerical accuracy, where F is the **CDF** of f and u, a given probability in range $(0, 1)$. We can use bisection method in this case because $F(x)$ is non-decreasing.

```r
inverse_F<-function(u,a, mu_1, mu_2, s_1, s_2,eps)
{
  if (u>=1 || u<=0) {stop("Error message: the value of CDF should be in ]0,1[")}

  x1 = -1 #initialize two values of x
  x2 = 1
  i=1

  #The while loop expands the bounds (x1 and x2) until F(x1) < u < F(x2),
  while((F(a, mu_1, mu_2, s_1, s_2, x1)-u)>0 || (F(a, mu_1, mu_2, s_1, s_2, x2)-u)<0){
    if(F(a, mu_1, mu_2, s_1, s_2, x1)-u>0){
       x1 <- x1*(2^i) # Expand x1 in the negative direction
    }
    if(F(a, mu_1, mu_2, s_1, s_2, x2)-u<0){
       x2 <- x2*(2^i)  # Expand x2 in the positive direction
    }
    i <- i+1
  }

  x = x1

  while(abs(x1-x2)>eps){
    x = (x1+x2)/2  # Calculate midpoint
    if((F(a, mu_1, mu_2, s_1, s_2, x)-u)<0){
      x1 <-x # Narrow down the interval
    }else{
      x2<-x
    }
  }
  return(x)

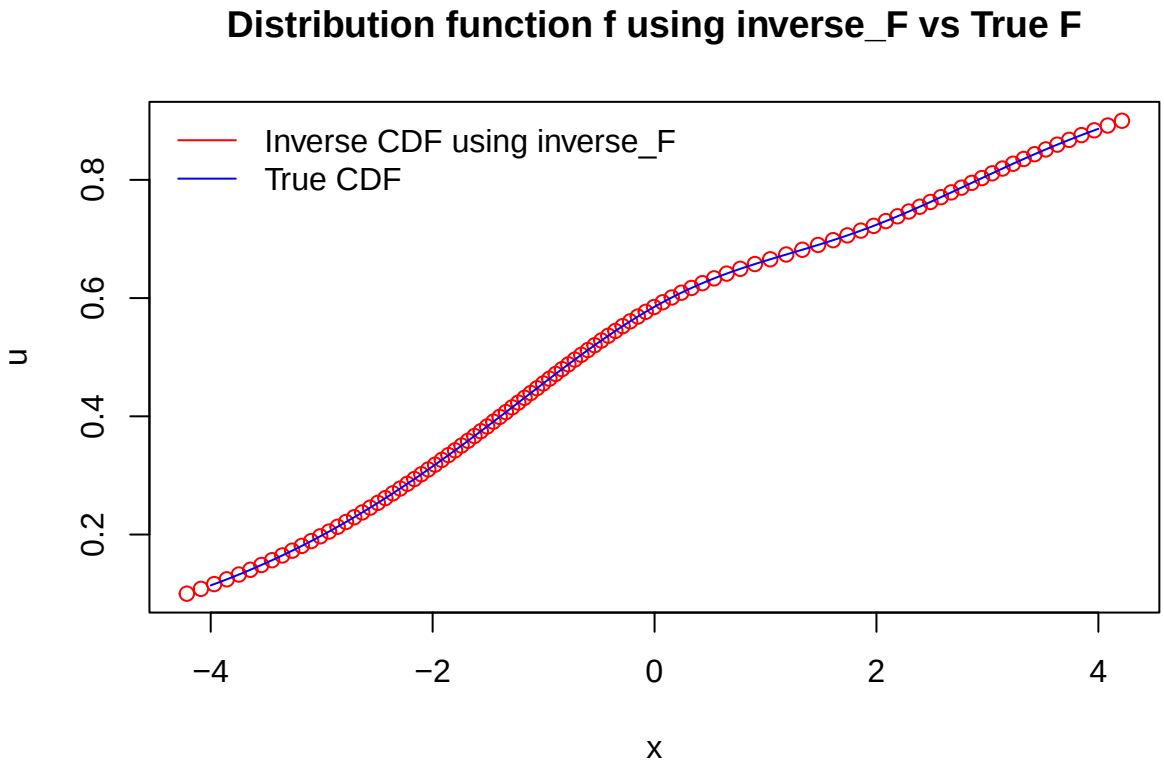
}

# Test the inverse function method
mu_1 = 0
mu_2 = 1
s_1 = 3
s_2 = 1
a = 0.2

# F_mirror wraps inverse_F, fixing parameters so we can apply it to multiple u values easily
F_mirror<-function(u){
  return(inverse_F(u,a, mu_1, mu_2, s_1, s_2,eps=0.0001))
}
u<-seq(0.1,0.9,length.out=100) #create a sequence of u

x <- sapply(u,F_mirror) #apply F_mirror to each element of u to get the root x
z<-seq(-4,4,length.out = 1000)
```

```r
y <- F(a, mu_1, mu_2, s_1, s_2, z) # apply F to get the real CDF value as comparison
plot(x,u,col="red", main = "Distribution function f using inverse_F vs True F")
lines(z,y,col="blue")
legend("topleft", legend = c("Inverse CDF using inverse_F", "True CDF"),
       col = c("red", "blue"), lty = 1, bty = "n")
```



The inverse function method algorithm

The inverse function method generates random samples from a distribution by applying the inverse_F to uniformly distributed random numbers.

```r
X_inverse<-function(a, mu_1, mu_2, s_1, s_2,eps=0.0001){
  u<-runif(1) #generate a uniformly distributed random number
  return(inverse_F(u,a, mu_1, mu_2, s_1, s_2,eps))
}
```

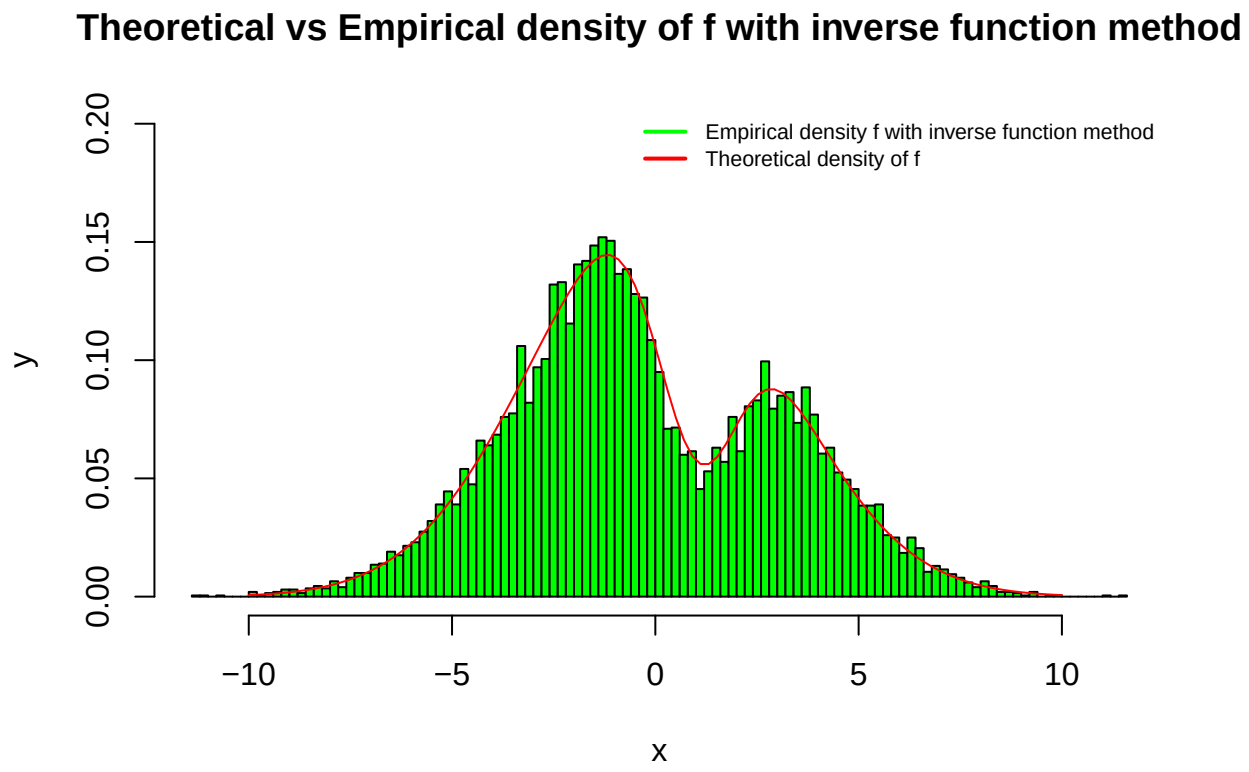
Below is the inv_cdf(n) that generates n samples from f using the inverse function method.

```r
inv_cdf<-function(n){
  X<-numeric(length = n) #Initialize a numeric vector to store n samples
  for (i in 1:n){
    X[i]<-X_inverse(a, mu_1, mu_2, s_1, s_2,eps=0.0001) # Generate and store each sample
  }
  return(X)
```

```r
}

#Test
mu_1 = 0
mu_2 = 1
s_1 = 3
s_2 = 1
a = 0.2
n = 10000
set.seed(1)

# Plot the histogram of the empirical density using the inverse function method
hist(inv_cdf(n),breaks = 100, col = 'green', ylim = c(0,0.2), freq = FALSE,
     c(5, 4, 4, 8),
     main = "Theoretical vs Empirical density of f with inverse function method", xlab="x",ylab="y")
x2<-seq(-10, 10,length.out = 100)
y2<-f(a, mu_1, mu_2, s_1, s_2, x2)# Calculate the theoretical density of f at each x value
lines(x2,y2, col='red') # Add the theoretical density as a red line on the plot
legend("topright", legend = c("Empirical density f with inverse function method",
                              "Theoretical density of f"),
       col = c("green", "red"), lwd =2, cex= 0.7, bty="n")
```



We observe that the empirical density f generated by the inverse function method aligns well with the theoretical density of f.

Accept-Reject Random Variable simulation.

Question 6

Since the density of f is known, we can apply the **Accept-Reject method**, which involves selecting a candidate density g. In this case, we choose f_1 as g, and set $M = \frac{1}{1-a}$. We have verified that the following conditions are satisfied:

1. $\frac{f(x)}{g(x)} \leq M$ for all x.
2. f and g have compatible supports.

Algorithm:

1. Generate a random variable Y from the distribution of g, and a random variable U from Uniform$(0, 1)$.
2. Accept $X = Y$ if $U \leq \frac{f(Y)}{Mg(Y)}$.
3. If the condition in step 2 is not satisfied, return to step 1.

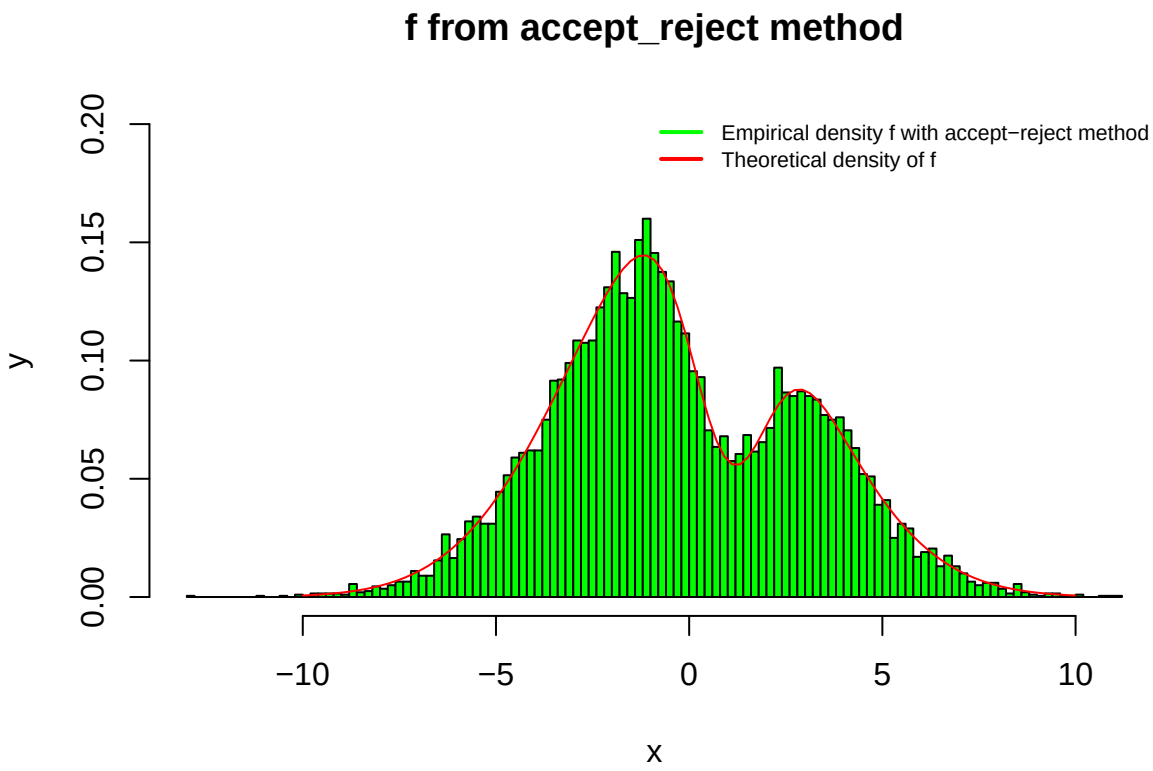
Question 7

```r
#The accept_reject_once function generate one sample following f using the accept-reject algorithm.
accept_reject_once<-function(a, mu_1, mu_2, s_1, s_2){
  M<- 1/(1-a)
  #Generate a candidate sample from the selected g (which is f1, a normal distribution)
  y<- rnorm(1,mean = mu_1, sd = s_1)
  u<- runif(1)
  i = 1 # Initialize iteration counter to track number of tries
  #Keep generating candidates until one is accepted
  while((M*u)>(f(a, mu_1, mu_2, s_1, s_2, y)/dnorm(y, mu_1, s_1))){
    u<-runif(1)
    y<-rnorm(1,mean = mu_1, sd = s_1)
    i <- i+1
  }
  return (list(rv = y, nt = i))# Return the accepted sample and the number of tries
}

#The accept_reject function generates n samples from the target distribution f
accept_reject<-function(n){
  X<-numeric(n) # Initialize a vector to store n accepted samples
  s<- 0 # Initialize a counter to sum up the total number of iterations (trials)
  for (j in 1:n){
      # Generate one accepted sample using accept-reject
      res <- accept_reject_once(a, mu_1, mu_2, s_1, s_2)
      X[j]<- res$rv # Store the accepted sample in the vector
      s <- s+res$nt  # Accumulate the number of iterations for acceptance
    }
  return (list(rv = X, ar =n/s) ) # Return the accepted samples and the acceptance rate
}

#Test
mu_1 = 0
```

```r
mu_2 = 1
s_1 = 3
s_2 = 1
a = 0.2
n = 10000
set.seed(1)
res<-accept_reject(n)
hist(res$rv,freq=FALSE, col='green', breaks = 100, ylim = c(0,0.2),
     main = "f from accept_reject method", xlab="x",ylab="y")
x2<-seq(-10, 10,length.out = 100)
y2<-f(a, mu_1, mu_2, s_1, s_2, x2)
lines(x2,y2, col='red')
legend("topright", legend = c("Empirical density f with accept-reject method",
                              "Theoretical density of f"),
       col = c("green", "red"), lwd =2, cex= 0.7, bty="n")
```



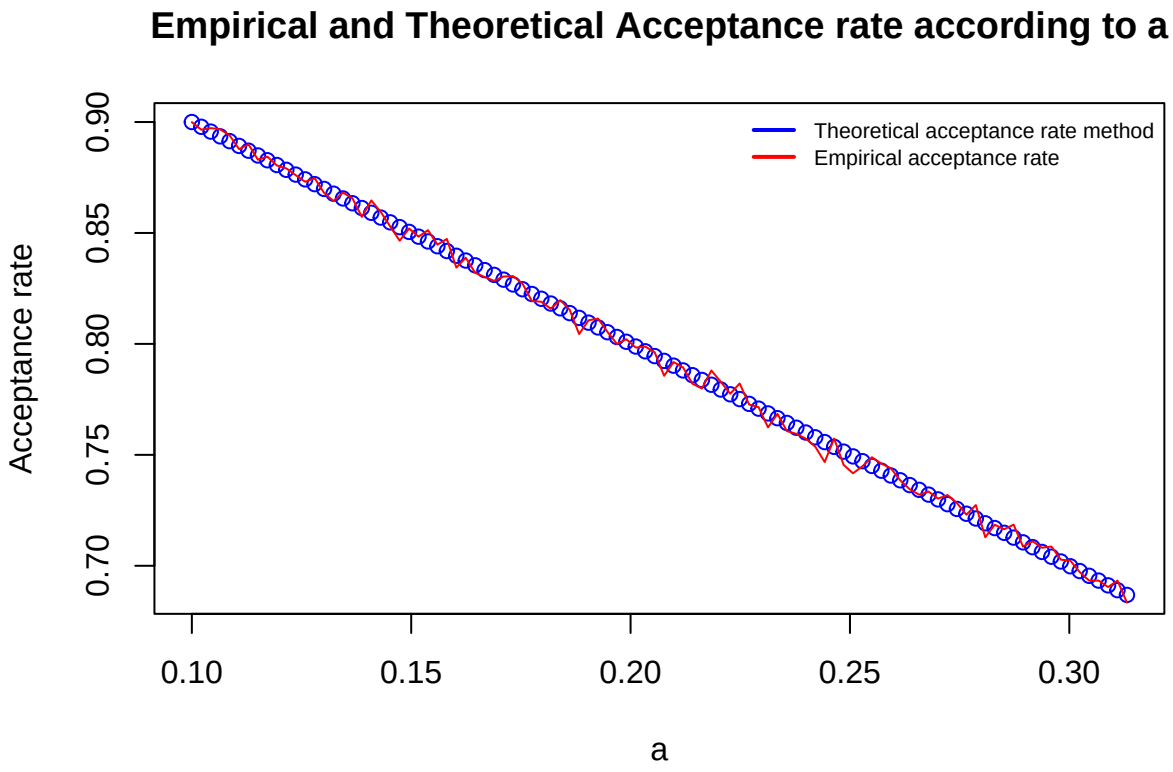
We observe that the empirical density f generated by the accept-reject method aligns well with the theoretical density of f.

Question 8

```r
b = seq(0.1,a_borne,length.out=100) # Create a sequence of a values ranging from 0.1 to a_borne
t_ar = 1-b # Calculate the theoretical acceptance rate as 1 - b

```

```r
# Plot the theoretical acceptance rate
plot(b,t_ar, col = "blue", xlab="a", ylab="Acceptance rate",
     main="Empirical and Theoretical Acceptance rate according to a")
ar_list<-numeric(100)# Initialize a numeric vector to store empirical acceptance rates
j = 1
set.seed(1)
for (a in b){
  res<-accept_reject(10000)# Generate 10,000 samples using accept-reject method
  ar_list[j]<-res$ar  # Store the empirical acceptance rate of each sample
  j <- j+1
}
lines(b,ar_list,col="red") # Plot the empirical acceptance rate
legend("topright", legend = c("Theoretical acceptance rate method","Empirical acceptance rate"),
       col = c("blue", "red"), lwd =2, cex= 0.7, bty="n")
```



We observe that the empirical acceptance rate closely matches the theoretical acceptance rate. As a approaches $a*$, the acceptance rate decreases, reducing the efficiency of the algorithm.
